Azure Multi-Service Cloud Application

Programming Assignment 4

Cloud Development

Total Points: 100

Contents

1 Overview

1.1 Mission

Deploy **TaskFlow**, an event-driven cloud pipeline on Microsoft Azure. A user submits an order through a web UI; a Durable Function orchestrates the workflow; a validation microservice on Azure Kubernetes Service (AKS) approves the order; a report-generator job on Container Instances produces a PDF and writes it to blob storage. The focus is **not** on writing application code: starter code is provided. The focus is on understanding *when to choose each Azure compute service* and how to wire them together securely.

Each service in this assignment has a deliberate, non-interchangeable role:

- **App Service** hosts a stateful, long-running web UI with CI/CD from GitHub
- **Durable Functions** coordinates a multi-step async workflow and persists state between steps
- **Azure Kubernetes Service (AKS)** runs a long-lived HTTP microservice with declarative infrastructure
- **Container Instances** runs a short-lived batch job that starts, does work, and exits: no idle cost
- **Container Registry** is the single source of truth for all images

You could not cleanly swap AKS and ACI in this architecture, and the write-up at the end asks you to explain why.

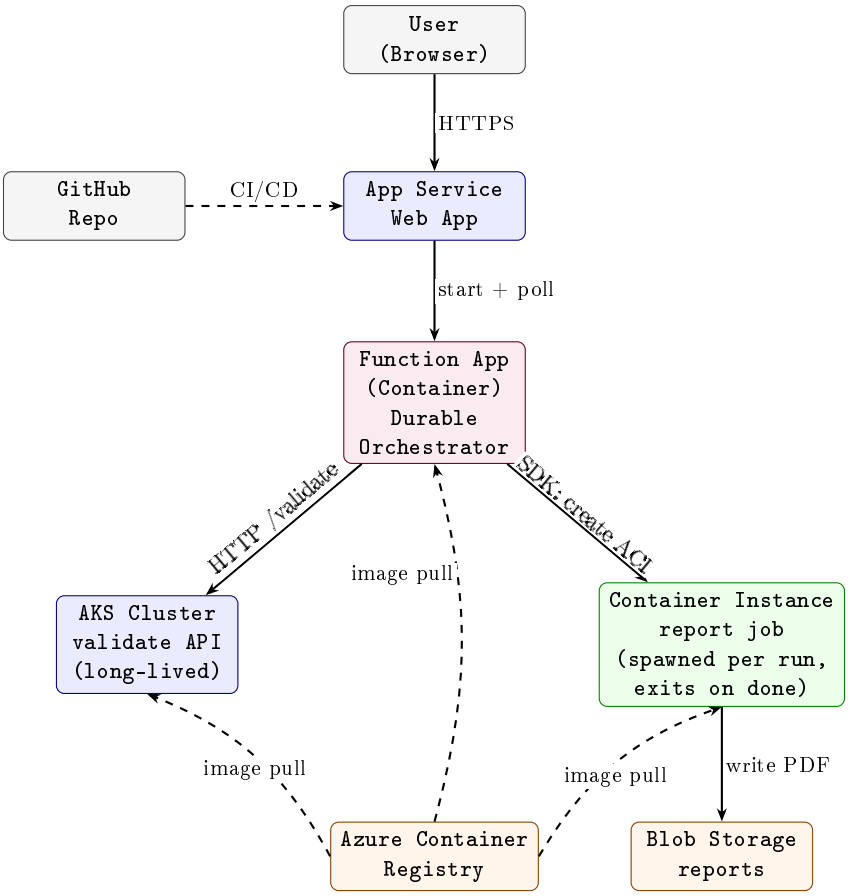
1.2 Learning Objectives

Upon completing this assignment, you will be able to:

- Deploy a web application from GitHub to Azure App Service
- Build and push Docker container images to Azure Container Registry (ACR)
- Deploy a containerised Azure Function App with a Durable Function workflow
- Deploy containers using Azure Container Instances (ACI)
- Deploy and manage containers using Azure Kubernetes Service (AKS)
- Build a simple distributed cloud architecture where a frontend integrates with multiple backend services

1.3 High-Level Architecture

The diagram below shows how all the components connect in the final deployed solution.



Component	Azure Service	Role
Web Dashboard	App Service (Web App)	Frontend UI; deployed from GitHub via CI/CD
Orchestrator	Azure Functions (containerised, Durable)	Kicks off the pipeline, calls AKS validator, then spawns the ACI report job
Validator	Azure Container App	Long-lived HTTPS microservice with scale-to-zero; validates orders
Report Job	Azure Container Instance	Created per run by the orchestrator; writes a PDF to blob, exits
Report Storage	Blob Storage `reports` container	Where the report job writes its PDF output
Image Registry	Azure Container Registry	Stores all container images

1.4 Prerequisites

- **Instructor-Provided Azure Access**: All necessary roles have been granted to your account, scoped to your assigned Resource Group. You will perform all tasks inside this Resource Group.
- Docker Desktop installed locally (version 24+ recommended)
- Azure CLI installed (version 2.50+ required)
- Azure Functions Core Tools (version 4.x required)

- Git installed for source code management
- Access to Azure Portal: `https://portal.azure.com`

Important

Budget & Cleanup: You are using the instructor's subscription. You **must** clean up resources when not actively working. Delete any manual test containers and scale deployments down when done to prevent exhausting the course budget.

Important

Region Strategy: To prevent quota limits in the instructor's subscription, you must use the following region based on your Roll Number:

- If your roll number starts with **2027-10-xxxx**: Deploy ALL resources to **UK West** (`ukwest`).
- If your roll number starts with **2023-xx-xxxx** or **2024-xx-xxxx** or **2025-xx-xxxx** or **2026-10-xxxx**: Deploy ALL resources to **UAE North** (`uaenorth`). If you encounter quota limits in UAE North, you may use **UK South** (`uksouth`) or **Sweden Central** (`swedencentral`) as a fallback.

Use your assigned region consistently for all resources. Mixing regions causes deployment failures.

2 Resource Naming Convention

Replace `<rollnum>` with your actual roll number (e.g., `27100445`). ACR names must be **alphanumeric only** (no hyphens) and globally unique. All other names allow hyphens.

Resource	**Name**
Resource Group	`rg-sp26-<rollnum>`
Container Registry	`pa4<rollnum>`
App Service Plan	`pa4-<rollnum>`
Web App	`pa4-<rollnum>`
Function App	`pa4-<rollnum>`
Container Apps Env	`pa4-<rollnum>`
Container App (validator)	`pa4-<rollnum>`
Container Instance pattern	`pa4-<rollnum>` (spawned per run)
Storage Account	`pa4<rollnum>` (for Functions + reports blob)
Blob Container	`reports`
Validator API Image	`pa4<rollnum>.azurecr.io/validate-api:v1`
Report Job Image	`pa4<rollnum>.azurecr.io/report-job:v1`
Function Image	`pa4<rollnum>.azurecr.io/func-app:v1`

Hint

ACR names only allow letters and numbers (5–50 characters). The prefix `cr` follows Microsoft's Cloud Adoption Framework naming convention. Web Apps and Function Apps are globally unique DNS names.

PART A: Web App Deployment from GitHub (15 Points)

3 Task 1: Deploy Web App from GitHub (15 Points)

3.1 Objective

Create your resource group and deploy the web application frontend to Azure App Service directly from your GitHub repository. The web app will serve as the single entry point for users to trigger all three backend services.

3.2 Step 1.1: Fork & Clone the Starter Repository

1. Go to the starter repository (link provided on LMS) and click **Fork** at the top right
2. Clone your fork locally:

```
git clone https://github.com/KarmaMS/CS487-PA4.git
cd CS487-PA4
```

3. Review the project structure:

```
pa4-starter/
|-- webapp/          # Web app frontend (Node.js / Express)
|-- function-app/    # Durable Function code + Dockerfile
|-- validate-api/    # AKS long-lived validator (FastAPI)
|-- report-job/      # ACI one-shot report generator
+-- README.md
```

4. The `webapp/` folder contains a one-button form that POSTs orders to the Durable Function
5. The `function-app/` folder contains a Durable Functions skeleton. You will complete the TODO sections in `function_app.py`
6. The `validate-api/` image is deployed to AKS (Task 5)
7. The `report-job/` image is deployed on-demand to Container Instances (Task 6)

Hint

You need a GitHub account (free). Sign up at https://github.com/join if you do not have one. After forking, all your changes are pushed to **your own fork**, not the original repository.

3.3 Step 1.2: Log in to Azure CLI

Ensure you are logged into the correct tenant and subscription:

```
az login
az account show --output table
```

3.4 App Service Plan Specifications

Parameter	Value
Name	`pa4-<rollnum>`
Resource Group	`rg-sp26-<rollnum>`
Region	UAE North or UK West (based on roll number)
OS	Linux
SKU	B1 (Basic)

3.5 Web App Specifications

Parameter	Value
Name	`pa4-<rollnum>`
Resource Group	`rg-sp26-<rollnum>`
App Service Plan	`pa4-<rollnum>`
Runtime Stack	Node 20 LTS
Deployment Source	GitHub (your forked repository)

3.6 Step 1.3: Create App Service & Configure Deployment

1. Create the App Service Plan and Web App
2. Configure deployment from GitHub using the **Deployment Center** in the Azure Portal, or use GitHub Actions
3. Select your forked repository and the `main` branch
4. Verify deployment triggers automatically

Hint

You can configure GitHub deployment via the Portal (**Deployment Center** → **GitHub**) which auto-generates a GitHub Actions workflow file, or configure it manually. Both approaches are acceptable.

3.7 Step 1.4: Configure Application Settings

Add the following Application Settings via **Settings** → **Environment variables**. You will populate the actual values in later tasks:

Setting	Description
`FUNCTION_START_URL`	Durable Function HTTP starter URL with function key (set in Task 4)
`FUNCTION_STATUS_URL`	Durable Function status query URL prefix (set in Task 4)

Hint

In this PA the Web App talks only to the Durable Function. The AKS validator and ACI report job are invoked by the Durable Function itself: the Web App never calls them directly. This is the correct architectural pattern: the frontend has one backend contract, and the orchestration layer hides the rest.

3.8 Step 1.5: Verify Deployment

1. Navigate to `https://pa4-<rollnum>.azurewebsites.net`
2. Confirm the dashboard loads with a **Submit Order** form and a **Status** panel
3. Submitting the form will return an error at this stage since the Durable Function is not yet deployed (this is expected)

Success Criteria

Expected result: The web app loads over HTTPS and displays the TaskFlow order form. Submitting shows "Function not configured" or a 500 error until Task 4 is complete.

3.9 Deliverables

- Screenshot of your forked GitHub repository
- Screenshot of Web App overview showing **Running** status
- Screenshot of Deployment Center showing GitHub configuration
- Screenshot of the dashboard loading in a browser
- Screenshot of Application Settings configured

PART B: Azure Container Registry (15 Points)

4 Task 2: Create ACR & Push Images (15 Points)

4.1 Objective

Create your own Azure Container Registry, build Docker images for (a) the validator API, (b) the report-job, and (c) the Function App, tag them, push them to ACR, and verify. These images are consumed by AKS (Task 5), ACI (Task 6), and the Function App (Task 4) respectively.

4.2 ACR Specifications

Parameter	Value
Name	`pa4<rollnum>`
Resource Group	`rg-sp26-<rollnum>`
Region	UAE North or UK West (based on roll number)
SKU	Basic
Admin User	Enabled

4.3 Step 2.1: Provision ACR

1. Create the Azure Container Registry with the specifications above
2. Enable the Admin user (Settings → Access keys)
3. Verify the ACR is visible in the portal with status **Succeeded**

4.4 Step 2.2: Review & Complete Dockerfiles

1. Review `validate-api/Dockerfile` (builds the FastAPI validator container, port 8080)
2. Review `report-job/Dockerfile` (builds the one-shot report-generator container)
3. Review `function-app/Dockerfile` (builds the Durable Function container from the Azure Functions Python base image)
4. Complete any missing instructions in the Dockerfiles based on the `README.md`

Hint

The validator exposes `POST /validate` and `GET /health`. The report-job has no HTTP endpoint: it reads env vars, generates a PDF, writes it to blob, and exits. The Function App base image is `mcr.microsoft.com/azure-functions/python:4-python3.11`. See the starter README for image-specific guidance.

4.5 Step 2.3: Build & Test Locally

```
# Build the validator API image
docker build -t validate-api:latest ./validate-api

# Build the report-job image
docker build -t report-job:latest ./report-job

# Build the Function App image
docker build -t func-app:latest ./function-app
```

```
# Test the validator locally
docker run --rm -p 8080:8080 validate-api:latest
# In another terminal:
curl -X POST http://localhost:8080/validate \
  -H "Content-Type: application/json" \
  -d '{"order_id":"LOCAL-1","items":[{"sku":"ABC","qty":2}]}'
```

Hint

On Apple Silicon Macs, build with `-platform linux/amd64` or Azure will refuse the image:

```
docker build --platform linux/amd64 -t validate-api:latest ./validate-api
```

Apply the same flag to all three builds.

4.6 Step 2.4: Tag & Push to ACR

```
# Log in to ACR
az acr login --name pa4<rollnum>

# Tag images
docker tag validate-api:latest pa4<rollnum>.azurecr.io/validate-api:v1
docker tag report-job:latest pa4<rollnum>.azurecr.io/report-job:v1
docker tag func-app:latest pa4<rollnum>.azurecr.io/func-app:v1

# Push images
docker push pa4<rollnum>.azurecr.io/validate-api:v1
docker push pa4<rollnum>.azurecr.io/report-job:v1
docker push pa4<rollnum>.azurecr.io/func-app:v1

# Verify
az acr repository list --name pa4<rollnum> --output table
```

4.7 Deliverables

- Screenshot of ACR overview in the portal showing **Succeeded**
- Screenshot of successful Docker builds for all three images
- Screenshot of local test of the validator (`POST /validate` returning JSON)
- Screenshot of successful pushes to ACR for all three images
- Screenshot of ACR repository list showing `validate-api`, `report-job`, and `func-app`

PART C: Azure Functions & Durable Workflow (20 Points)

5 Task 3: Implement the Durable Orchestrator (12 Points)

5.1 Objective

Implement the Durable orchestrator that coordinates the full TaskFlow pipeline. The orchestrator receives an order from the Web App, calls the validator running on Container Apps, and (if validated) spawns a Container Instance to generate a report. The starter code provides the skeleton; you will fill in the activity bodies.

5.2 Orchestrator Flow

1. **HTTP Starter** receives an order payload (`{order_id, items}`) from the Web App
2. **Orchestrator** invokes `validate_activity`
3. **Activity 1:** `validate_activity` makes an HTTPS call to the AKS validator; returns `{valid: true/false, reason: ...}`
4. If invalid, orchestrator returns `{status: rejected}` and stops
5. If valid, orchestrator invokes `report_activity`
6. **Activity 2:** `report_activity` uses the Azure SDK to create a new Container Instance running the report-job image; waits for it to reach `Succeeded`; returns the blob URL of the generated report
7. Orchestrator returns `{status: completed, report_url: ...}`

5.3 Durable Function Components

Component	Role
HTTP Starter	Receives the order POST and starts the orchestrator
Orchestrator	Calls validate_activity, branches on result, then calls report_activity
`validate_activity`	HTTP call to AKS `/validate` endpoint
`report_activity`	Uses `ContainerInstanceManagementClient` to create an ACI, poll until done, return blob URL

5.4 Step 3.1: Implement the Activities

The starter `function-app/function_app.py` contains skeletons for both activities marked with `TODO`. Your job:

`validate_activity` (~5 lines):

1. Read `VALIDATE_URL` from environment variables
2. POST the order payload as JSON
3. Return the parsed JSON response

`report_activity` (~25 lines, mostly boilerplate provided):

1. Read the required environment variables (provided as constants in the skeleton)
2. Use `ContainerInstanceManagementClient` from the Azure SDK (import already provided) to create a container group with:

- Image: from `REPORT_IMAGE` env var
- Registry credentials: from `ACR_USERNAME` / `ACR_PASSWORD`
- Environment variables: the order payload and `STORAGE_CONN` so the container can write output
- Restart policy: `Never`
- Resources: 1 vCPU, 1.5 GiB

3. Poll the container group until `instance_view.state == "Succeeded"` (or fail after 5 minutes)
4. Clean up the ACI (`begin_delete`) so it stops billing after the work is done
5. Return the expected blob URL `https://stpa4<roll>.blob.core.windows.net/reports/<order_id>.pd`

Important

The SDK boilerplate (imports, client construction, container group template) is provided in the skeleton. You are filling in the order-specific fields, not writing SDK code from scratch. Read the skeleton carefully before typing anything. If something in the skeleton is unclear, the Azure SDK docs for `ContainerInstanceManagementClient` are your authority: https://learn.microsoft.com/en-us/python/api/azure-mgmt-containerinstance.

5.5 Step 3.2: Implement the Orchestrator

Wire the two activities together. Pseudocode:

```
@app.orchestration_trigger(context_name="context")
def my_orchestrator(context):
    order = context.get_input()
    validation = yield context.call_activity("validate_activity", order)
    if not validation.get("valid"):
        return {"status": "rejected", "reason": validation.get("reason", "unknown")}
    report_url = yield context.call_activity("report_activity", order)
    return {"status": "completed", "report_url": report_url}
```

Important

The chained pattern above (activity A → conditional → activity B) is the simplest demonstration of Durable Functions' core value: the orchestrator checkpoints state between activities, so if activity 2 fails and the runtime replays the orchestrator, activity 1 does not re-execute. A plain HTTP function cannot do this.

5.6 Step 3.3: Test Locally (Partial)

Local testing is limited for this PA because `report_activity` needs Azure credentials to create an ACI: which you cannot do from a laptop without first authenticating. You **can** test `validate_activity` locally once you deploy the AKS validator (Task 5) and export its URL. End-to-end testing happens after Tasks 4–6.

```
cd function-app
pip install -r requirements.txt
# Start Azurite for Durable storage backend in another terminal: ‘azurite‘
export VALIDATE_URL=http://<aks-service-external-ip>:8080/validate   # after Task 5
func start
```

Then trigger the orchestrator:

```
curl -X POST http://localhost:7071/api/orchestrators/my_orchestrator \
  -H "Content-Type: application/json" \
  -d '{"order_id":"LOCAL-1","items":[{"sku":"ABC","qty":2}]}'
```

Expect the response to include a `statusQueryGetUri`. The orchestration will reach `validate_activity`, succeed if the AKS validator is up, and then fail at `report_activity` (because you do not have cloud credentials locally). This is expected.

Hint

Install Azurite with `npm install -g azurite`. Set `AzureWebJobsStorage` to `UseDevelopmentStorage=true` in `local.settings.json` so Durable Functions can use Azurite as its state backend.

5.7 Deliverables

- Your completed `function_app.py` (in the submitted GitHub repo)
- Screenshot of `func start` showing all four Durable handlers registered (starter, orchestrator, validate_activity, report_activity)
- Screenshot of a local smoke test of `validate_activity` against your deployed AKS service: only after Task 5. If you reach here before Task 5, skip this screenshot and produce it during Task 7

6 Task 4: Deploy Function App as Container (8 Points)

6.1 Objective

Create a Function App resource on Azure and deploy it using the updated container image. Since you wrote new code in Task 3, you must rebuild and push the image before deploying it.

6.2 Function App Specifications

Parameter	Value
Name	`pa4-<rollnum>`
Resource Group	`rg-sp26-<rollnum>`
Region	UAE North or UK West (based on roll number)
Hosting Plan	App Service plan (reuse existing `pa4-<rollnum>`)
Image Source	ACR: `pa4<rollnum>.azurecr.io/func-app:v1`
Runtime	Python 3.11

6.3 Step 4.1: Rebuild and Push the Function App Image

Because you completed the `TODO`s in `function_app.py` during Task 3, the image you pushed in Task 2 is now outdated.

```
cd function-app
docker build --platform linux/amd64 -t func-app:latest .
docker tag func-app:latest pa4<rollnum>.azurecr.io/func-app:v1
docker push pa4<rollnum>.azurecr.io/func-app:v1
```

6.4 Step 4.2: Deploy Function App

1. Create the Function App in the Azure Portal or via CLI, selecting your ACR image as the deployment source
2. Configure the Function App to pull the container image from your ACR
3. Verify the Function App starts successfully
4. Retrieve the HTTP starter endpoint URL

Hint

When creating a containerised Function App, select **Container Image** as the publish method in the Portal (instead of "Code"). Point it to your ACR image. The Function App needs a Storage Account; one will be auto-created or you can specify your own (use `pa4<rollnum>` as defined in the naming table).
Choose **App Service plan** as the hosting plan, and select the exact same `pa4-<rollnum>` plan you created in Task 1. This allows your Function App to run on the same underlying VM as your Web App, which avoids extra costs and fully supports custom Docker containers (unlike the Consumption plans).

6.5 Step 4.3: Smoke-Test the Deployed Function

Retrieve the function key (needed to invoke the HTTP starter):

```
az functionapp function keys list \
  --name pa4-<rollnum> \
  --resource-group rg-sp26-<rollnum> \
```

```
  --function-name http_starter \
  --query default -o tsv
```

Now trigger the orchestrator directly:

```
curl -X POST \
  "https://pa4-<rollnum>.azurewebsites.net/api/orchestrators/my_orchestrator?code=<KEY>" \
  -H "Content-Type: application/json" \
  -d '{"order_id": "SMOKE-001", "items": [{"sku": "ABC", "qty": 2}]}'
```

The response includes a `statusQueryGetUri`. Open it in a browser. The orchestration will show `Running` and then `Failed` (not `Completed`): the orchestrator tries to call `VALIDATE_URL`, which is not yet set. That is **expected at this stage**. It proves:

1. Your function code deployed successfully
2. The orchestrator started and checkpointed
3. The activity ran far enough to attempt the HTTP call

You will fix the failure by completing Tasks 5 and 6 and then running the end-to-end test in Task 7.

Success Criteria

Expected result at this stage: The `curl` returns an `id`, `statusQueryGetUri`, and other management URLs. Polling the status URL shows `runtimeStatus: Running` briefly, then `Failed` with an error message about being unable to reach `VALIDATE_URL`. This failure is the correct checkpoint: it means the deployment works end to end, only the downstream service is missing.

6.6 Deliverables

- Screenshot of Function App in Azure Portal showing the container image configuration
- Screenshot of the Functions list in the Portal showing `http_starter`, `my_orchestrator`, `validate_activity`, `report_activity`
- Screenshot of the `curl` output showing the orchestration started (instance id and `statusQueryGetUri` visible)
- Screenshot of the status query URL JSON showing `Failed` status at this stage

PART D: Container Deployments (30 Points)

7 Task 5: Azure Kubernetes Service (Validator) (15 Points)

7.1 Objective

Deploy the validator microservice to Azure Kubernetes Service (AKS). AKS is a fully managed Kubernetes cluster. It is significantly more complex than Container Apps, requiring explicit deployment configurations and load balancer setup, but it provides ultimate control over your container orchestration.

Important

Why AKS for this workload? The validator is a long-running HTTP service that gets called during every order. While Container Apps (Serverless) is easier, AKS represents the industry standard for enterprise microservice orchestration. You will deploy a `Standard_B2s` node to keep costs low.

7.2 AKS Cluster Specifications

Parameter	Value
Name	`pa4-<rollnum>`
Resource Group	`rg-sp26-<rollnum>`
Node Count	1
Node Size	`Standard_B2s`

7.3 Step 5.1: Create Cluster & Connect

Creating an AKS cluster takes 5–10 minutes. Run the following:

```
# Create the AKS cluster (this takes time)
az aks create \
  --resource-group rg-sp26-<rollnum> \
  --name pa4-<rollnum> \
  --node-count 1 \
  --node-vm-size Standard_B2s \
  --generate-ssh-keys

# Get credentials to configure kubectl
az aks get-credentials --resource-group rg-sp26-<rollnum> --name pa4-<rollnum>

# Verify connection
kubectl get nodes
```

7.4 Step 5.2: Create ACR Pull Secret

Because you do not have permission to create Role Assignments in the subscription, you cannot use the standard managed identity ACR attachment. Instead, you will create a Kubernetes Secret using your ACR admin credentials.

```
# Fetch ACR credentials
ACR_USER=$(az acr credential show -n pa4<rollnum> --query username -o tsv)
ACR_PASS=$(az acr credential show -n pa4<rollnum> --query "passwords[0].value" -o tsv)

# Create the secret in AKS
kubectl create secret docker-registry acr-secret \
  --docker-server=pa4<rollnum>.azurecr.io \
```

```
  --docker-username=$ACR_USER \
  --docker-password=$ACR_PASS
```

7.5 Step 5.3: Apply Kubernetes Configuration

In your starter repository, navigate to `validate-api/k8s/`. You will find `deployment.yaml` and `service.yaml`.

Edit `deployment.yaml`: Replace `<rollnum>` in the `image:` field with your actual roll number. Then apply both files to your cluster:

```
kubectl apply -f validate-api/k8s/deployment.yaml
kubectl apply -f validate-api/k8s/service.yaml

# Watch the pods start
kubectl get pods -w
```

7.6 Step 5.4: Verify the Validator Endpoint

The `validate-service` is of type `LoadBalancer`, which tells Azure to provision a Public IP. This takes 1–2 minutes.

```
# Get the EXTERNAL-IP (wait until it changes from <pending> to an IP address)
kubectl get service validate-service
```

Once you have the `EXTERNAL-IP`, verify the service:

```
# Export the IP (replace with your actual IP)
export IP="20.xxx.xxx.xxx"
echo "Validator URL: http://$IP:8080"

# Health check
curl http://$IP:8080/health

# Validate a good order (should return valid: true)
curl -X POST http://$IP:8080/validate \
  -H "Content-Type: application/json" \
  -d '{"order_id":"O-1001","items":[{"sku":"ABC","qty":2}]}'

# Validate a bad order (should return valid: false)
curl -X POST http://$IP:8080/validate \
  -H "Content-Type: application/json" \
  -d '{"order_id":"O-1002","items":[{"sku":"ABC","qty":999}]}'
```

Success Criteria

Expected results:

```
# Good order
{"valid": true, "reason": "ok", "order_id": "O-1001"}

# Bad order (qty > 100)
{"valid": false, "reason": "quantity exceeds limit", "order_id": "O-1002"}
```

7.7 Step 5.5: Wire the Function App

Give the Durable Function the validator URL so `validate_activity` can reach it:

```
az functionapp config appsettings set \
  --name pa4-<rollnum> \
  --resource-group rg-sp26-<rollnum> \
  --settings VALIDATE_URL="http://$IP:8080/validate"
```

7.8 Deliverables (Add to SUBMISSION.md)

- Screenshot of `kubectl get nodes` showing your cluster is running
- Screenshot of `kubectl get pods` showing the validator pod is `Running`
- Screenshot of `kubectl get service` showing the assigned `EXTERNAL-IP`
- Screenshot of `curl /health` and both `curl /validate` runs
- Screenshot of Function App Application Setting `VALIDATE_URL`

8 Task 6: Azure Container Instance (Report Job) (15 Points)

8.1 Objective

Deploy and test the report-generator container on Azure Container Instances. Unlike Task 5's AKS cluster, this workload has no persistent endpoint: it is a **one-shot job**. The Durable Function from Task 3 creates the ACI programmatically at run-time using the Azure SDK. In this task you will: (a) manually test the ACI lifecycle once using the CLI, (b) grant the Function App permission to create ACIs on its own, and (c) configure the settings the Function needs at run-time.

Important

Why ACI for this workload? AKS is already running the validator because it needs a stable service endpoint, but the report generator is a short-lived batch task. ACI bills only while the container is alive. A report generator that runs for 20 seconds once per order belongs on ACI: keeping this as another always-running Kubernetes workload would waste compute.

8.2 Step 6.1: Create the Blob Container for Reports

The report job writes its output PDF here:

```
az storage container create \
  --account-name pa4<rollnum> \
  --name reports \
  --auth-mode login
```

Hint

If this command fails with a permission error, your user account needs the **Storage Blob Data Contributor** role on the storage account. Assign it via `az role assignment create` or through the IAM blade in the Portal.

8.3 Step 6.2: Manual ACI Test (one-shot lifecycle)

Before letting the Function App create ACIs automatically, run one manually so you understand the lifecycle:

```
# Get ACR credentials
ACR_USER=$(az acr credential show -n pa4<rollnum> --query username -o tsv)
ACR_PASS=$(az acr credential show -n pa4<rollnum> --query "passwords[0].value" -o tsv)

# Get Managed Identity details
MI_CLIENT_ID=$(az identity show --name mi-pa4-<rollnum> --resource-group rg-sp26-<rollnum>
    --query clientId -o tsv)
MI_RESOURCE_ID=$(az identity show --name mi-pa4-<rollnum> --resource-group rg-sp26-<rollnum>
     --query id -o tsv)

# Create one ACI manually, running the report-job image
az container create \
  --resource-group rg-sp26-<rollnum> \
  --name ci-report-test \
  --image pa4<rollnum>.azurecr.io/report-job:v1 \
  --registry-login-server pa4<rollnum>.azurecr.io \
  --registry-username $ACR_USER \
  --registry-password $ACR_PASS \
  --assign-identity $MI_RESOURCE_ID \
```

```
  --cpu 1 --memory 1.5 \
  --restart-policy Never \
  --environment-variables \
      ORDER_ID=TEST-001 \
      ORDER_JSON='{"order_id":"TEST-001","items":[{"sku":"ABC","qty":2}]}' \
      STORAGE_ACCOUNT_URL="https://pa4<rollnum>.blob.core.windows.net" \
      AZURE_CLIENT_ID="$MI_CLIENT_ID"

# Watch it run and exit
az container show --resource-group rg-sp26-<rollnum> \
  --name ci-report-test \
  --query "instanceView.state" --output tsv
# Expect: Running -> Succeeded (or Terminated)

# View logs from the container
az container logs --resource-group rg-sp26-<rollnum> --name ci-report-test

# Confirm the PDF landed in blob storage
az storage blob list --account-name pa4<rollnum> --container-name reports \
  --auth-mode login --output table
```

Success Criteria

Expected result: The ACI transitions to `Succeeded`, its logs show PDF generation and an upload line, and `az storage blob list` shows `TEST-001.pdf` in the `reports` container.

8.4 Step 6.3: Attach the Pre-Provisioned Managed Identity

The Durable Function's `report_activity` creates ACIs on demand. Instead of a secret, it uses a Managed Identity. Your instructor has already provisioned an identity named `mi-pa4-<rollnum>` in your resource group and granted it the necessary permissions.

You must now attach this identity to your Function App:

1. In the Azure Portal, go to your **Function App** → **Settings** → **Identity**.
2. Select the **User assigned** tab (next to System assigned).
3. Click + **Add**.
4. Select your subscription and the identity named `mi-pa4-<rollnum>`.
5. Click **Add**.

Hint

Managed identity means the Function App authenticates to Azure without any secret stored in code or config. `DefaultAzureCredential` in the Python SDK picks up the identity automatically at runtime. This is the canonical way Azure services talk to each other: worth understanding well beyond this course.

8.5 Step 6.4: Configure Function App Settings

The `report_activity` needs several values at runtime. Set them all:

```
ACR_USER=$(az acr credential show -n pa4<rollnum> --query username -o tsv)
ACR_PASS=$(az acr credential show -n pa4<rollnum> --query "passwords[0].value" -o tsv)
SUB_ID=$(az account show --query id -o tsv)
MI_CLIENT_ID=$(az identity show --name mi-pa4-<rollnum> --resource-group rg-sp26-<rollnum>
    --query clientId -o tsv)
```

```
az functionapp config appsettings set \
  --name pa4-<rollnum> \
  --resource-group rg-sp26-<rollnum> \
  --settings \
    REPORT_IMAGE="pa4<rollnum>.azurecr.io/report-job:v1" \
    ACR_SERVER="pa4<rollnum>.azurecr.io" \
    ACR_USERNAME="$ACR_USER" \
    ACR_PASSWORD="$ACR_PASS" \
    STORAGE_ACCOUNT_URL="https://pa4<rollnum>.blob.core.windows.net" \
    REPORT_RG="rg-sp26-<rollnum>" \
    REPORT_LOCATION="<uaenorth-or-ukwest>" \
    SUBSCRIPTION_ID="$SUB_ID" \
    AZURE_CLIENT_ID="$MI_CLIENT_ID"
```

8.6 Step 6.5: Cleanup the Test ACI

Delete the manual test container so it doesn't sit as a stopped resource cluttering your resource group:

```
az container delete --resource-group rg-sp26-<rollnum> --name ci-report-test --yes
```

8.7 Deliverables

- Screenshot of the `reports` blob container after creation
- Screenshot of `az container show` output with `state: Succeeded` after the manual run
- Screenshot of `az container logs` showing the report-job's own output (PDF generation lines)
- Screenshot of the generated PDF listed in the `reports` blob container
- Screenshot of the Function App's Identity blade showing the **User-assigned** identity (`mi-pa4-<rollnum>`) attached
- Screenshot of Function App Application Settings showing the `REPORT_*`, `ACR_*`, and `STORAGE_ACCOUNT_URL` values (mask the passwords)

PART E: Integration & Documentation (15 Points)

9 Task 7: End-to-End Pipeline Test (15 Points)

9.1 Objective

Demonstrate a single user action through the web UI that triggers the full pipeline: Web App → Durable Function (HTTP starter) → orchestrator → `validate_activity` (calls ACA) → `report_activity` (creates ACI) → blob write → status polling → report URL displayed in UI.

9.2 Step 7.1: Wire the Web App to the Function

```
FUNC_BASE="https://pa4-<rollnum>.azurewebsites.net"
FUNC_KEY=$(az functionapp function keys list \
  --name pa4-<rollnum> \
  --resource-group rg-sp26-<rollnum> \
  --function-name http_starter \
  --query default -o tsv)

az webapp config appsettings set \
  --name pa4-<rollnum> \
  --resource-group rg-sp26-<rollnum> \
  --settings \
    FUNCTION_START_URL="$FUNC_BASE/api/orchestrators/my_orchestrator?code=$FUNC_KEY" \
    FUNCTION_STATUS_URL="$FUNC_BASE/runtime/webhooks/durabletask/instances"
```

9.3 Step 7.2: Happy Path (Submit a Valid Order)

Open the live Web App and submit an order with `qty=2` and any SKU. Capture screenshots at each stage:

1. Form filled, before submit
2. Status panel showing `Running` with a live instance ID
3. Status panel showing `Completed` with the report URL link
4. The downloaded PDF open in a viewer

Also capture evidence that each service participated in the run:

5. Function App **Monitor** → **Invocations** showing the orchestration run and both activities
6. `az container list` output showing the ACI that was spawned (its name will follow the pattern `ci-report-<order_id>`)
7. The blob container showing the new PDF matching your order ID
8. AKS pod logs or AKS metrics showing the validator received traffic during the run

9.4 Step 7.3: Reject Path (Submit an Invalid Order)

Submit an order that the validator will reject. The starter validator rejects orders with `qty > 100`. Capture:

1. UI showing the rejection message and reason
2. `az container list` output showing **no** new ACI was created for this order (proving the orchestrator short-circuited correctly)

3. Function App **Monitor** showing the orchestration completed `Successfully` but with `status: rejected` as the output

Success Criteria

Expected result: A valid order produces a PDF in blob storage within ~60 seconds of submission. An invalid order is rejected by the validator and **no ACI is created**. Both paths are visible in the Function App's invocation logs.

9.5 Step 7.4: Resource Group Overview

Navigate to your resource group in the Azure Portal and take a screenshot showing **all deployed resources**:

- App Service Plan, Web App
- Function App + Storage Account (`pa4<rollnum>`)
- Container Registry (`pa4<rollnum>`)
- AKS cluster (`pa4-<rollnum>`) and its managed node resource group
- (Optional) A stopped ACI if your happy-path run left one behind

9.6 Deliverables

- The four happy-path screenshots from Step 7.2
- The four participation-evidence screenshots from Step 7.2
- The three rejection-path screenshots from Step 7.3
- Screenshot of resource group showing all deployed resources

10 Task 8: Documentation & Architecture Diagram (5 Points)

10.1 Report Requirements (2–3 Pages)

Include a short write-up covering:

1. **Service selection: explain the choices.** For each of App Service, Durable Functions, Azure Kubernetes Service (AKS), and Container Instances, write 3–4 sentences explaining why it is the right tool for its specific role in TaskFlow. Be specific about cost, scale behaviour, and operational model.
2. **ACI vs. AKS: hands-on comparison.** Using your AKS cluster metrics and the `az container list` screenshot from Task 7.2, explain:
   - What happens to AKS when it is idle for 10 minutes?
   - What does "idle" even mean for ACI in your pipeline?
   - If a malicious user spammed the Submit button 1000 times in a minute, which service would incur the most cost, and why?
3. **Durable Functions: why not just HTTP?** Your orchestrator chains `validate` and `report`, and the report step takes up to a minute. Explain in 3–4 sentences what would be harder if you implemented the same flow as two plain HTTP-triggered functions that called each other. Name at least two concrete problems (hint: function timeouts, state persistence, retry-on-failure).
4. **Cost review.** Screenshot **Cost Management → Cost Analysis** scoped to your resource group. Estimate what this PA consumed in the instructor's subscription. Identify the single most expensive resource.
5. **Challenges faced.** Describe at least two issues you hit and how you debugged them. The grader values honesty over "everything worked."

10.2 Architecture Diagram Requirements

Your diagram must show:

- All Azure resources with service names
- GitHub → App Service CI/CD flow
- Web App → Function App (start + status polling)
- Function App → AKS Cluster (validate HTTP call)
- Function App → Container Instance (SDK-based creation) labelled "ephemeral, per run"
- Container Instance → Blob Storage (PDF write)
- ACR providing images to Function App, AKS, and ACI
- Managed identity / IAM relationship between Function App and the resource group

Recommended tools: Draw.io, Lucidchart, Microsoft Visio, or hand-drawn and scanned.

10.3 Deliverables (Commit to GitHub)

- Write-up embedded into `SUBMISSION.md` in your repository
- Architecture diagram committed to the `docs/` folder of your repository

11 Final Submission Package

Submission Requirements

Your entire submission is managed through your GitHub repository. You must submit **ONLY ONE ITEM** on LMS:

1. **GitHub Repository URL**: Your forked repository containing all code, Dockerfiles, and your completed `SUBMISSION.md` file.

Important Warning: Your last commit time on GitHub will be tracked and used to determine any late penalties. Ensure all your work is committed and pushed before the deadline.
Inside your repository: You must commit to your forked repo all necessary submission materials. Copy `SUBMISSION_TEMPLATE.md` to `SUBMISSION.md`, fill in all the required screenshots and written answers, and push it.

Important

Screenshot Requirements: Since you are working in assigned resource groups within the instructor's subscription, screenshots are the **primary grading evidence**. Each screenshot **must** include:

- The **Azure Portal URL bar** or CLI output visible (proves you are using your assigned resource group)
- **Resource names** matching your roll number (e.g., `rg-sp26-27100445`)
- **Timestamps** visible where possible (browser tab, CLI output, Azure activity log)
- **Brief captions** describing what each screenshot shows

Screenshots that do not clearly show ownership may receive **zero marks** for that task.

12 Grading Rubric

Criteria	Points
PART A: Web App & GitHub Deployment	**15**
Task 1: App Service + GitHub Actions CI/CD working	15
PART B: Azure Container Registry	**15**
Task 2: ACR + 3 images (validator, report-job, func-app) pushed	15
PART C: Azure Functions + Durable Orchestration	**20**
Task 3: Orchestrator + 2 activities implemented correctly	12
Task 4: Containerised Function App deployed from ACR	8
PART D: Container Deployments	**30**
Task 5: AKS validator + Kubernetes service observed	15
Task 6: ACI report job + managed identity + permissions	15
PART E: Integration & Documentation	**20**
Task 7: End-to-end pipeline (happy + reject paths)	15
Task 8: Write-up + architecture diagram	5
Late Policy	
Within 6 hours late	−10%
Within 24 hours late	−20%
After 24 hours	−100%
Deductions	
Missing screenshots	−2 each
Total	**100**

13 Important Guidelines

Important

- **This is a capstone, not a simple integration**: The architecture forces each service into a distinct role. Before submitting, make sure you can explain *why* each service is where it is.
- **Follow naming conventions exactly**: Grading depends on consistent resource naming.
- **Use the smallest paid tier that works**: B1 App Service, one `Standard_B2s` AKS node, Basic ACR, and one-shot ACI. Never pick P1/P2/Premium or multi-node AKS.
- **Expected spend:** The AKS node is the main cost driver. A one-week run is reasonable for the course subscription, but leaving clusters after grading is not acceptable.
- **Clean up when not working**: Delete stopped ACIs. Delete the AKS cluster after final screenshots if instructed. Orchestrator-spawned ACIs should be auto-deleted by your `report_activity`.
- **Screenshot everything**: Both successful and failed operations. The `Failed` orchestration after Task 4 is a *deliverable*, not a mistake.
- **Monitor your spending**: Check Cost Management in the Azure Portal at least every other day.
- **Role assignments take 5–10 minutes**: If a 403 appears right after granting a role, wait before panicking.

14 Troubleshooting

Issue	Solution
Web App shows "Application Error"	Check Deployment Center → Logs. Verify runtime stack and Application Settings.
GitHub Actions workflow fails	Check the workflow YAML for correct `app-name`. Verify the publish profile secret is set in GitHub → Settings → Secrets.
ACR pull fails (401)	Verify Admin user is enabled. Check credentials with `az acr credential show`.
`func start` fails locally	Ensure Azure Functions Core Tools v4 is installed. Install Azurite for local storage emulation. Check Python version.
Durable orchestration stuck	Verify `AzureWebJobsStorage` is set. Check the Storage Account is accessible.
ACI container not starting	Check container logs: `az container logs`. Verify the image exists in ACR and credentials are correct.
AKS service stuck at `EXTERNAL-IP <pending>`	Wait a few minutes. If it remains pending, check public IP quota in the assigned region and verify the service type is `LoadBalancer`.
Function App returns 500 when calling `report_activity`	User-assigned managed identity (`mi-pa4-<rollnum>`) not attached to the Function App, or `REPORT_*` app settings not populated. Verify the identity is attached in the Portal Identity blade.
ACI creation fails with `InaccessibleImage`	Either ACR admin user disabled, or wrong credentials in the SDK call. Re-run `az acr credential show` and update Function App settings. If credentials are correct, wait 30–60s after `docker push` before retrying: ACR needs a moment to serve the new image.
Orchestration never reaches `report_activity`	Validator returned `valid: false`. Check the order payload: the starter validator rejects on `qty > 100` and unknown SKUs. This is the *correct* behaviour for the reject-path test.
Report PDF never appears in blob	The ACI ran but could not write. Check `az container logs` for the failed run. Usually `STORAGE_CONN` is wrong or the `reports` container doesn't exist.
Durable status stuck at `Running` forever	The activity threw an unhandled exception. Open Function App **Monitor** → **Invocations** → click the failed activity for the full stack trace.
403 Forbidden creating ACI from Function	Role assignment hasn't propagated yet. Wait another 5 minutes and retry. Do not re-assign the role: that makes no difference.
Region quota error	Do not switch regions on your own. Screenshot the error and contact the instructor; the course split between `uaenorth` and `ukwest` is intended to avoid AKS and ACI capacity limits.
Unexpected cost increase	The usual cause is an AKS cluster left running. Delete unused resources and report any resource outside your assigned group.

15 Resource Cleanup

Important

You are responsible for your assigned Azure resources. When you are done with the assignment and have taken final screenshots:

1. Take a final screenshot of your resource group showing all resources
2. Delete your AKS cluster to prevent ongoing charges:

```
az aks delete --name pa4-<rollnum> --resource-group rg-sp26-<rollnum> --yes --no
    -wait
```

3. **DO NOT** delete the entire resource group. Your instructor may verify your deployment during grading.

While working: Delete the manual test Container Instance from Task 6.2 after you verify it worked. Orchestrator-spawned ACIs are auto-deleted by `report_activity` at the end of each run, so those do not need manual cleanup. The AKS node keeps billing while the cluster exists.

16 Learning Resources

16.1 Azure Labs & Tutorials

- **App Service Quickstart:** https://learn.microsoft.com/en-us/azure/app-service/quickstart-nodejs
- **Deploy to App Service using GitHub Actions:** https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions
- **ACR Quickstart (Push an image):** https://learn.microsoft.com/en-us/azure/container-registry/container-registry-get-started-docker-cli
- **Durable Functions Quickstart (Python):** https://learn.microsoft.com/en-us/azure/azure-functions/durable/quickstart-python-vscode
- **Durable Functions Patterns:** https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview?tabs=python
- **Deploy a Function App from Container:** https://learn.microsoft.com/en-us/azure/azure-functions/functions-deploy-container
- **ACI Quickstart (Deploy a container):** https://learn.microsoft.com/en-us/azure/container-instances/container-instances-quickstart
- **AKS Quickstart:** https://learn.microsoft.com/en-us/azure/aks/learn/quick-kubernetes-depl
- **AKS LoadBalancer Services:** https://learn.microsoft.com/en-us/azure/aks/load-balancer-st

16.2 Essential Documentation

- App Service: https://learn.microsoft.com/en-us/azure/app-service/
- Azure Functions: https://learn.microsoft.com/en-us/azure/azure-functions/
- Durable Functions: https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview
- Azure Kubernetes Service: https://learn.microsoft.com/en-us/azure/aks/
- Container Instances: https://learn.microsoft.com/en-us/azure/container-instances/
- ACR: https://learn.microsoft.com/en-us/azure/container-registry/

16.3 Key Concepts

App Service: Fully managed PaaS for hosting web apps. Supports deployment from GitHub with CI/CD via GitHub Actions or the Deployment Center.

Azure Functions (Durable): Event-driven serverless compute. Durable Functions extend Functions with stateful workflows: an HTTP Starter triggers an Orchestrator that coordinates Activity Functions. Can be deployed as code or as a container.

Azure Container Instances: The simplest way to run a container in Azure. No server management, no orchestration. Just deploy and run. Best for simple, standalone containers that need a public IP.

Azure Kubernetes Service: Managed Kubernetes for running containerized workloads with declarative manifests, pods, services, and cluster-level operations. Best when you need Kubernetes primitives or want to learn production orchestration tradeoffs.

Azure Container Registry: Private Docker registry in Azure. Stores your container images and provides them to ACI, AKS, and Function Apps during deployment.